

AI ASSISTED CODING ASSIGNMENT – 8.1

CH. NAKSHATRA (2303A51094)

BATCH-02

Task 1 (Password Strength Validator – Apply AI in Security Context)

Task: Apply AI to generate at least 3 assert test cases for `is_strong_password(password)` and implement the validator function

Requirements:

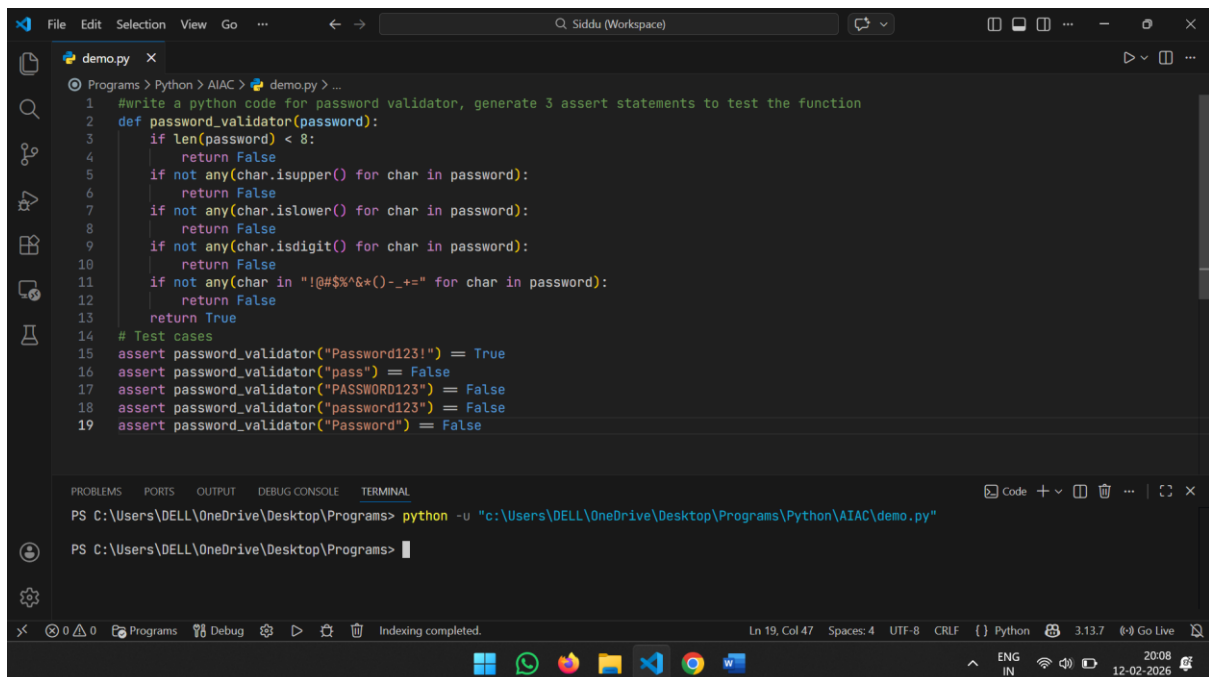
- Password must have at least 8 characters.
- Must include uppercase, lowercase, digit, and special character.
- Must not contain spaces.

Example Assert Test Cases:

- `assert is_strong_password("Abcd@123") == True`
- `assert is_strong_password("abcd123") == False`
- `assert is_strong_password("ABCD@1234") == True`

Expected Output:

- Password validation logic passing all AI-generated test cases.



The screenshot shows a VS Code editor window with a file named `demo.py`. The code defines a `password_validator(password)` function that checks for password strength based on the requirements. It includes several conditional checks for length, character types, and special characters. Below the function, there are five assert test cases. The terminal at the bottom shows the command to run the script: `python -u "c:\Users\DELL\OneDrive\Desktop\Programs\Python\AIAC\demo.py"`.

```
1 #write a python code for password validator, generate 3 assert statements to test the function
2 def password_validator(password):
3     if len(password) < 8:
4         return False
5     if not any(char.isupper() for char in password):
6         return False
7     if not any(char.islower() for char in password):
8         return False
9     if not any(char.isdigit() for char in password):
10        return False
11    if not any(char in "!@#$%^&*()-_+=" for char in password):
12        return False
13    return True
14 # Test cases
15 assert password_validator("Password123!") == True
16 assert password_validator("pass") == False
17 assert password_validator("PASSWORD123") == False
18 assert password_validator("password123") == False
19 assert password_validator("Password") == False
```

PROBLEMS PORTS OUTPUT DEBUG CONSOLE TERMINAL

PS C:\Users\DELL\OneDrive\Desktop\Programs> python -u "c:\Users\DELL\OneDrive\Desktop\Programs\Python\AIAC\demo.py"

PS C:\Users\DELL\OneDrive\Desktop\Programs> |

Task 2 (Number Classification with Loops – Apply AI for Edge Case Handling)

Task: Use AI to generate at least 3 assert test cases for a `classify_number(n)` function. Implement using loops.

Requirements:

- Classify numbers as Positive, Negative, or Zero.
- Handle invalid inputs like strings and None.
- Include boundary conditions (-1, 0, 1).

Example Assert Test Cases:

- `assert classify_number(10) == "Positive"`
- `assert classify_number(-5) == "Negative"`
- `assert classify_number(0) == "Zero"`

Expected Output:

- Classification logic passing all assert tests.

The screenshot shows a VS Code editor window with a file named `demo.py`. The code defines a `classify_number` function and includes three assert statements to test it. The terminal output shows that all test cases passed.

```

1 #write a python to classify numbers into positive, negative, or zero, generate 3 assert statements with boundries to test the function
2 def classify_number(num):
3     if num > 0:
4         return "positive"
5     elif num < 0:
6         return "negative"
7     else:
8         return "zero"
9 # Test cases
10 assert classify_number(5) == "positive", "Test case 1 failed"
11 assert classify_number(-3) == "negative", "Test case 2 failed"
12 assert classify_number(0) == "zero", "Test case 3 failed"
13 print("All test cases passed!")
14

```

Terminal Output:

```

PS C:\Users\DELL\OneDrive\Desktop\Programs> python -u "c:\Users\DELL\OneDrive\Desktop\Programs\Python\AIAC\demo.py"
All test cases passed!
PS C:\Users\DELL\OneDrive\Desktop\Programs>

```

Task 3 (Anagram Checker – Apply AI for String Analysis)

Task: Use AI to generate at least 3 assert test cases for `is_anagram(str1, str2)` and implement the function.

Requirements:

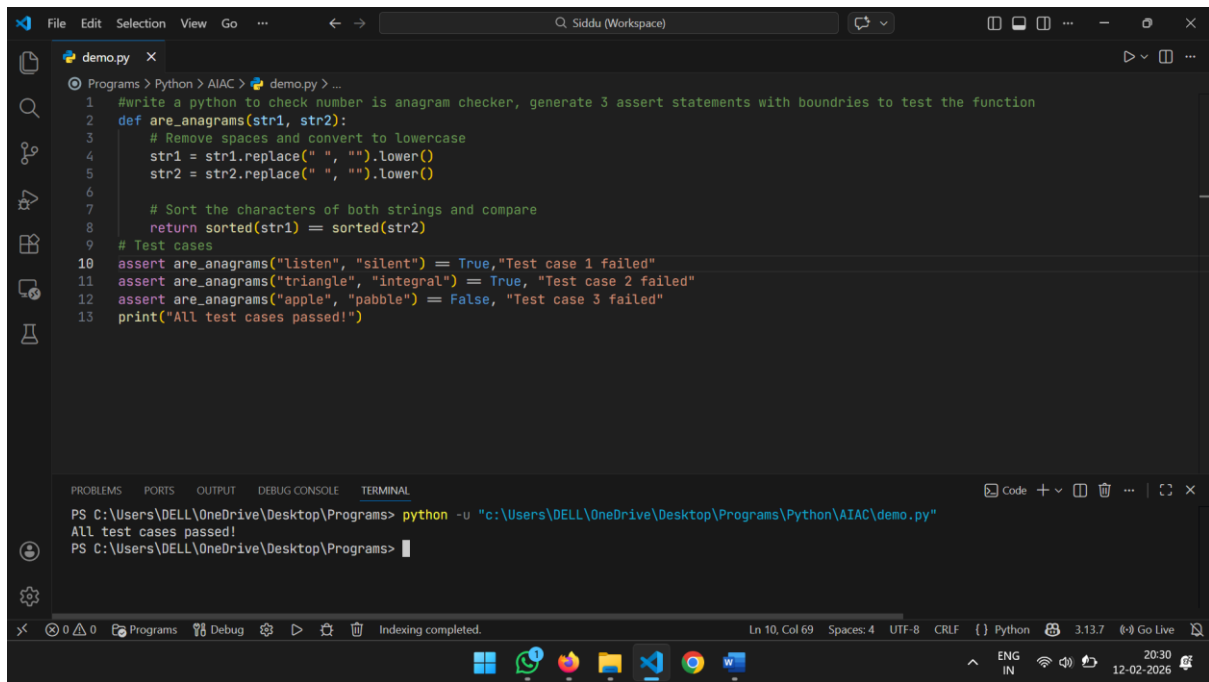
- Ignore case, spaces, and punctuation.
- Handle edge cases (empty strings, identical words).

Example Assert Test Cases:

- `assert is_anagram("listen", "silent") == True`
- `assert is_anagram("hello", "world") == False`
- `assert is_anagram("Dormitory", "Dirty Room") == True`

Expected Output:

- Function correctly identifying anagrams and passing all AI-generated tests.



```
demo.py
1 #Write a python to check number is anagram checker, generate 3 assert statements with boundries to test the function
2 def are_anagrams(str1, str2):
3     # Remove spaces and convert to lowercase
4     str1 = str1.replace(" ", "").lower()
5     str2 = str2.replace(" ", "").lower()
6
7     # Sort the characters of both strings and compare
8     return sorted(str1) == sorted(str2)
9
10 # Test cases
11 assert are_anagrams("listen", "silent") == True, "Test case 1 failed"
12 assert are_anagrams("triangle", "integral") == True, "Test case 2 failed"
13 assert are_anagrams("apple", "pabble") == False, "Test case 3 failed"
14 print("All test cases passed!")

PS C:\Users\DELL\OneDrive\Desktop\Programs> python -u "c:\Users\DELL\OneDrive\Desktop\Programs\Python\AIAC\demo.py"
All test cases passed!
PS C:\Users\DELL\OneDrive\Desktop\Programs>
```

Task 4 (Inventory Class – Apply AI to Simulate Real-World Inventory System)

Task: Ask AI to generate at least 3 assert-based tests for an Inventory class with stock management.

Requirements:

- add_item(name, quantity)
- remove_item(name, quantity)
- get_stock(name)

Example Assert Test Cases:

- nv = Inventory()
- inv.add_item("Pen", 10)
- assert inv.get_stock("Pen") == 10
- inv.remove_item("Pen", 5)
- assert inv.get_stock("Pen") == 5
- inv.add_item("Book", 3)
- assert inv.get_stock("Book") == 3

Expected Output:

- Fully functional class passing all assertions.

```
1 # Task: Ask AI to generate at least 3 assert-based tests for an Inventory class with stock management.
2 class Inventory:
3     def __init__(self):
4         self.stock = {}
5     def add_item(self, name, quantity):
6         if name in self.stock:
7             self.stock[name] += quantity
8         else:
9             self.stock[name] = quantity
10    def remove_item(self, name, quantity):
11        if name in self.stock and self.stock[name] >= quantity:
12            self.stock[name] -= quantity
13        else:
14            raise ValueError("Not enough stock to remove")
15    def get_stock(self, name):
16        return self.stock.get(name, 0)
17
18 # Test Cases
19 inv = Inventory()
20 inv.add_item("Pen", 10)
21 assert inv.get_stock("Pen") == 10
22 inv.remove_item("Pen", 5)
23 assert inv.get_stock("Pen") == 5
24 inv.add_item("Book", 3)
25 assert inv.get_stock("Book") == 3
```

```
PS C:\Users\DELL\OneDrive\Desktop\Programs> python -u "c:\Users\DELL\OneDrive\Desktop\Programs\Python\AIAC\demo.py"
All tests passed successfully!
PS C:\Users\DELL\OneDrive\Desktop\Programs>
```

Task 5 (Date Validation & Formatting – Apply AI for Data Validation)

Task: Use AI to generate at least 3 assert test cases for `validate_and_format_date(date_str)` to check and convert dates.

Requirements:

- Validate "MM/DD/YYYY" format.
- Handle invalid dates.
- Convert valid dates to "YYYY-MM-DD"

Example Assert Test Cases:

- `assert validate_and_format_date("10/15/2023") == "2023-10-15"`
- `assert validate_and_format_date("02/30/2023") == "Invalid Date"`
- `assert validate_and_format_date("01/01/2024") == "2024-01-01"`

Expected Output:

- Function passes all AI-generated assertions and handles edge cases.

The image shows a Visual Studio Code editor window with a Python file named `demo.py` open. The code defines a function `validate_and_format_date` that takes a date string and returns it in a standard format. It uses a loop to try different date formats: `%Y-%m-%d`, `%d/%m/%Y`, and `%m-%d-%Y`. If none of these work, it raises a `ValueError` with the message "Date format not recognized". The function is tested with several assertions, and the results are printed.

```
4 def validate_and_format_date(date_str):
5     try:
6         for fmt in ('%Y-%m-%d', '%d/%m/%Y', '%m-%d-%Y'):
7             try:
8                 date_obj = datetime.strptime(date_str, fmt)
9                 return date_obj.strftime('%Y-%m-%d') # Return in standard format
10            except ValueError:
11                continue
12        raise ValueError("Date format not recognized")
13    except ValueError as e:
14        return str(e)
15
16 # Test cases
17 assert validate_and_format_date("2024-06-15") == "2024-06-15" # Valid ISO format
18 assert validate_and_format_date("15/06/2024") == "2024-06-15" # Valid European format
19 assert validate_and_format_date("06-15-2024") == "2024-06-15" # Valid US format
20 assert validate_and_format_date("2024/06/15") == "Date format not recognized" # Invalid format
21 assert validate_and_format_date("June 15, 2024") == "Date format not recognized" # Invalid format
22 assert validate_and_format_date("2024-15-06") == "Date format not recognized" # Invalid format
23 print("All test cases passed!")
```

The terminal output shows the command `python -u "c:\Users\DELL\OneDrive\Desktop\Programs\Python\AIAC\demo.py"` being executed, and the output is "All test cases passed!".

PS C:\Users\DELL\OneDrive\Desktop\Programs> python -u "c:\Users\DELL\OneDrive\Desktop\Programs\Python\AIAC\demo.py"
All test cases passed!
PS C:\Users\DELL\OneDrive\Desktop\Programs>